Kwikby REST API — Phase 0 and Phase 1 guide

Contents

1. What this API is . . . 4
   - Why stored procedures . . . 4
2. Solution layout . . . 4
   - Endpoint → procedure map . . . 5
3. Prerequisites . . . 5
4. Configuration . . . 5
   - Configuration sources and precedence . . . 6
   - Production JWT key rule . . . 6
5. Database setup . . . 6
   - 5.1 SQL Server 2014 compatibility . . . 6
   - 5.2 Procedure files . . . 7
   - 5.3 Deploying to a scratch database (required before live) . . . 7
   - 5.4 Deploying procedures to an existing database . . . 7
   - 5.5 SQL login for the API . . . 7
   - 5.6 Custom SQL error numbers . . . 8
6. Running the API . . . 8
   - 6.1 Development . . . 8
   - 6.2 Health responses . . . 8
7. Authentication and authorization . . . 9
   - 7.1 Getting a token . . . 9
   - 7.2 Claims . . . 9
   - 7.3 Roles versus vendor membership . . . 10
   - 7.4 Lockout and account enumeration . . . 10
   - 7.5 Token expiry and revocation . . . 10
8. Endpoint reference . . . 10
   - 8.1 Products — search . . . 10
   - 8.2 Products — detail . . . 11
   - 8.3 Categories . . . 12
   - 8.4 Shops . . . 12
   - 8.5 Platform dashboard (admin) . . . 13
   - 8.6 Shop dashboard (admin or shop member) . . . 13
9. Errors . . . 14
10. Security . . . 14
    - 10.1 Transport . . . 14
    - 10.2 Headers . . . 15
    - 10.3 CORS . . . 15
    - 10.4 Rate limiting . . . 15

10.5 SQL safety . . . 15
10.6 Secrets . . . 15
10.7 Swagger . . . 16
11. Testing . . . 16
11.1 Unit and contract tests . . . 16
11.2 Real-process smoke test . . . 16
11.3 CI . . . 17
12. Deployment . . . 17
12.1 Publish . . . 17
12.2 IIS . . . 17
12.3 Production checklist . . . 17
12.4 Rollback . . . 18
13. Troubleshooting . . . 18
14. Phase status and next steps . . . 18
15. Quick reference . . . 19
16. Phase 2 and Phase 3 endpoints (delivered) . . . 19
Catalogue (Phase 2) . . . 20
Shops and inventory (Phase 2) . . . 20
Commerce (Phase 3) . . . 20

> Status: **Phase 0 (foundation) and Phase 1 (read-only catalogue, authentication, > dashboards) are implemented and tested.** Phase 2 (write endpoints) and Phase 3 > (checkout, payment and ledger) are not part of this delivery. > > The existing MVC application is unchanged at runtime. The only edit to > `Kwikby_MP.csproj` is a set of `Remove` globs so the new `Kwikby_MP.Api` and > `Kwikby_MP.Shared` folders are not compiled into the MVC project.

---

1. What this API is

`Kwikby_MP.Api` is a separate ASP.NET Core 9 Web API for the LibanPost eShop marketplace.

- **Data access:** every request path calls a SQL Server stored procedure through

Dapper. There is no EF Core in the API.

- **Authentication:** JWT bearer tokens only. The MVC cookie scheme is not used.
- **Database:** the same SQL Server database as the MVC application (`MVStores` in

production). The API is read-only in this phase.

- **Shared code:** `Kwikby_MP.Shared` links the existing

`Services/PasswordHasher.cs`, so the API verifies exactly the same PBKDF2 hashes the MVC application created.

- **Independent deployment:** the API can be published, scaled and restarted without

touching the MVC site.

Why stored procedures

The marketplace already has working, unit-tested C# services for order placement, payments and the vendor ledger. Those stay in C# (Phase 3). Duplicating that logic in T-SQL would create two implementations that drift. Stored procedures are used for the paths that benefit from a single database round-trip and a stable contract: search, product detail, dashboards, authentication lookups and atomic stock changes.

---

2. Solution layout

```
Kwikby_MP.sln
Kwikby_MP.csproj Existing MVC application (unchanged code)
Kwikby_MP.Api/ New REST API
Controllers/ Auth, Products, Categories, Shops, Dashboard
Repositories/ One repository per read area
Infrastructure/ JWT, Dapper gateway, options, errors, health
Models/Requests|Responses/ DTOs and query strings
appsettings.json Empty connection string, safe defaults
appsettings.Development.json Development-only JWT key
Properties/launchSettings.json http://localhost:5269, Swagger on launch
Kwikby_MP.Shared/ Links Services/PasswordHasher.cs
Tests/Kwikby_MP.Api.Tests/ 45 unit + SQL contract tests
Database/
02_StoredProcedures.sql Search, detail, dashboards, stock adjustment
04_ApiStoredProcedures.sql Auth, category and shop procedures
05_Deploy.ps1 Idempotent procedure deployment
06_SeedApiTest.ps1 Safe scratch-database setup and seed
docs/
API_GUIDE.md This document
api-smoke.ps1 Real-process end-to-end smoke test
API_IMPLEMENTATION_PLAN.md Original plan and endpoint catalogue
```

Endpoint → procedure map

Endpoint	Stored procedure
`POST /api/v1/auth/token`	`usp_Auth_GetUserForLogin`, `usp_Auth_RecordLogin`, `usp_Auth_RecordFailedLogin`
`GET /api/v1/auth/me`	`usp_Auth_GetUserClaims`
`GET /api/v1/products`	`usp_SearchProducts`
`GET /api/v1/products/{id}`	`usp_GetProductDetail` (four result sets)
`GET /api/v1/categories`	`usp_Category_List`
`GET /api/v1/shops`	`usp_Shop_List`
`GET /api/v1/shops/{id}`	`usp_Shop_Get`
`GET /api/v1/admin/dashboard`	`usp_GetPlatformDashboard`
`GET /api/v1/shops/{id}/dashboard`	`usp_GetShopDashboard` (two result sets)
Phase 2 stock endpoint (not exposed yet)	`usp_AdjustStock`

3. Prerequisites

Requirement	Notes
.NET 9 SDK	The API and shared project target `net9.0`.
SQL Server 2014 or later	Tested on SQL Server 2014 SP2 Express, compatibility level 120.
Windows PowerShell 5.1	Used by `05_Deploy.ps1`, `06_SeedApiTest.ps1` and `api-smoke.ps1`.
SQL login with EXECUTE	The API login needs `CONNECT` and `EXECUTE` on the `usp_*` procedures. Ownership chaining covers the tables.

The API does **not** run EF Core migrations. The database schema is owned by the MVC application and `Database/00_Schema.sql`; the API only adds procedures.

4. Configuration

All API settings live under the `Api` section, except the connection string.

Key	Default	Purpose
`ConnectionStrings:MultiVendorsStoresConn`	empty	SQL Server connection string. Empty means the process starts but data endpoints return 503 and `/health/ready` is Unhealthy.
`Api:Enabled`	`true`	Maintenance switch. When `false`, every `/api/*` request returns 503 problem+json. Health and Swagger stay available.
`Api:CommandTimeoutSeconds`	`30`	Dapper command timeout for every stored procedure.
`Api:SlowCommandMs`	`1000`	A stored procedure slower than this logs a warning with its name and elapsed time.
`Api:Jwt:Issuer`	`kwikby_MP`	JWT `iss` claim and issuer validation.
`Api:Jwt:Audience`	`kwikby_MP.Api`	JWT `aud` claim and audience validation.
`Api:Jwt:SigningKey`	empty	HMAC-SHA256 signing key. **Must be at least 32 characters outside Development.**
`Api:Jwt:AccessTokenMinutes`	`60`	Access-token lifetime. There is no refresh token in Phase 1.
`Api:Cors:AllowedOrigins`	`[]`	Explicit browser origins. Empty means no cross-origin browser access.

Key	Default	Purpose
`Api:RateLimit:PermitLimit`	`100`	Requests per window, per token subject or anonymous IP.
`Api:RateLimit:WindowSeconds`	`60`	Fixed window length.

Configuration sources and precedence

ASP.NET Core reads, in order:

1. `appsettings.json`
2. `appsettings.{Environment}.json`
3. `appsettings.Local.json` (optional, loaded explicitly by `Program.cs`)
4. Environment variables
5. Command-line arguments

`appsettings.Local.json` is the recommended place for the SQL password on a developer machine. It is loaded from the API's content root; the repository root copy is used by the scripts. Never commit it.

Environment-variable form uses double underscores:

```
$env:ASPNETCORE_ENVIRONMENT = 'Production'
$env:ConnectionStrings__MultiVendorsStoresConn = 'Server=.;Database=MVStores;User
Id=kwikby_api;Password=...;Encrypt=True;TrustServerCertificate=True'
$env:Api__Jwt__SigningKey = '<at-least-32-random-characters>'
```

Production JWT key rule

`ApiOptions.Validate` throws at startup when `Api:Jwt:SigningKey` is missing or shorter than 32 characters and the environment is not `Development`. In Development it falls back to the public key `dev-only-kwikby-api-signing-key-change-me`; that key is deliberately obvious and is never accepted in Production.

---

5. Database setup

5.1 SQL Server 2014 compatibility

The live server is SQL Server 2014 (12.0, compatibility level 120). The scripts avoid features added after 2014:

- no `CREATE OR ALTER` (SQL Server 2016 SP1)
- no `DROP ... IF EXISTS`
- no `STRING_AGG`, `OPENJSON`, `JSON_VALUE`, `STRING_SPLIT`
- no `AT TIME ZONE`, `TRIM`, `CONCAT_WS`, `DATETRUNC`, `GREATEST`, `LEAST`

Every procedure uses the portable install pattern:

```
IF OBJECT_ID(N'dbo.usp_X', N'P') IS NULL
EXEC(N'CREATE PROCEDURE dbo.usp_X AS BEGIN SET NOCOUNT ON; END');
GO
ALTER PROCEDURE dbo.usp_X ...
GO
```

This is idempotent on SQL Server 2005 and later.

5.2 Procedure files

File	Procedures
`Database/02_StoredProcedures.sql`	`usp_SearchProducts`, `usp_GetProductDetail`, `usp_GetShopDashboard`, `usp_GetPlatformDashboard`, `usp_AdjustStock`
`Database/04_ApiStoredProcedures.sql`	`usp_Auth_GetUserForLogin`, `usp_Auth_GetUserClaims`, `usp_Auth_RecordLogin`, `usp_Auth_RecordFailedLogin`, `usp_Category_List`, `usp_Shop_List`, `usp_Shop_Get`

`Database/03_HeroEngine.sql` is deployed only when `05_Deploy.ps1` is called with `-IncludeHero`. The API does not use the hero procedures.

5.3 Deploying to a scratch database (required before live)

`06_SeedApiTest.ps1` creates or refreshes `MVStores_ApiTest`, applies the schema and procedures, and inserts a deterministic test dataset. It refuses to run unless the connection string's `Initial Catalog` is exactly `MVStores_ApiTest`.

```
cd <repository root>
./Database/06_SeedApiTest.ps1
```

It reads the SQL credentials from `appsettings.Local.json` (or `ConnectionStrings__MultiVendorsStoresConn`) and switches the catalog to the scratch database itself. You can also pass a full connection string:

```
./Database/06_SeedApiTest.ps1 -ConnectionString `
'Data Source=.;Initial Catalog=MVStores_ApiTest;Integrated
Security=True;TrustServerCertificate=True'
```

Expected output: 12 procedures and a seeded test shop/product/order.

5.4 Deploying procedures to an existing database

Review the script diff first, take a backup, then run:

```
./Database/05_Deploy.ps1 `
-ConnectionString 'Server=.;Database=MVStores;User
Id=...;Password=...;Encrypt=True;TrustServerCertificate=True'
```

The deployment script:

- reads the connection string from `-ConnectionString`, then

`ConnectionStrings__MultiVendorsStoresConn`, then `appsettings.Local.json`

- splits the script on real `GO` batch separators, ignoring `GO` inside block

comments, line comments and string literals

- executes every batch, stops on the first error, and reports which file/batch failed
- is idempotent: running it twice is safe

Do **not** point it at the live database until the scratch run and the SQL contract tests pass.

5.5 SQL login for the API

The API should not use `sa` or the MVC application's login. Create a dedicated login:

```
USE master;
CREATE LOGIN [kwikby_api] WITH PASSWORD = N'<strong password>';

USE MVStores;
CREATE USER [kwikby_api] FOR LOGIN [kwikby_api];
GRANT EXECUTE ON SCHEMA::dbo TO [kwikby_api];
```

`GRANT EXECUTE ON SCHEMA::dbo` is convenient because ownership chaining lets the procedures read the tables. If the database contains other procedures that the API must not call, grant `EXECUTE` on the twelve `usp_*` procedures individually instead.

5.6 Custom SQL error numbers

The procedures raise these numbers. `SqlErrorTranslator` maps them to safe HTTP responses and never leaks the raw SQL message for unexpected errors.

Number	Meaning	HTTP
`52001`	Validation / zero quantity	400
`52002`	Unknown enum value	400
`52003`	Entity not found	404
`52004`	Insufficient stock	409
`52005`	Duplicate slug / SKU / email	409
`52006`	Invalid state transition	409
`52007`	Forbidden shop / role scope	403
`52008`	Concurrency conflict	409
`2627`, `2601`	Unique key violation	409
`547`	Foreign-key violation	409
`1205`	Deadlock	503
`-2`	Command timeout	504
`4060`, `18456`, `233`, `10054`, `53`, `64`	Unreachable / login failure	503

---

6. Running the API

6.1 Development

```
cd <repository root>
$env:ASPNETCORE_ENVIRONMENT = 'Development'
$env:ConnectionStrings__MultiVendorsStoresConn = `
'Data Source=.;Initial Catalog=MVStores_ApiTest;Integrated
Security=True;TrustServerCertificate=True'
dotnet run --project Kwikby_MP.Api\Kwikby_MP.Api.csproj --urls http://localhost:5269
```

Then open:

URL	Purpose
`http://localhost:5269/swagger`	Swagger UI (Development only)
`http://localhost:5269/health/live`	Process liveness. Does not touch SQL.
`http://localhost:5269/health/ready`	SQL readiness. Returns 503 when the DB is unreachable.

The MVC application keeps running on port `5268`; the API uses `5269` so both can run side by side.

Swagger has a **Bearer** authorization button. Call `POST /api/v1/auth/token` first, copy the `accessToken` value, paste it into **Authorize**, and then try the protected endpoints. The token is validated with issuer, audience, signature and lifetime checks; clock skew is 2 minutes.

6.2 Health responses

`GET /health/live`:

```
{
"status": "Healthy",
"checks": []
```

```
}
```

`GET /health/ready` when SQL is reachable:

```
{
"status": "Healthy",
"checks": [
{ "name": "sqlserver", "status": "Healthy", "description": "SQL Server answered SELECT 1." }
]
}
```

When the connection string is missing or SQL is down, the status is `Unhealthy` and the HTTP status is `503`. The process still starts, so Swagger and the 503 problem responses remain available for diagnosis.

---

7. Authentication and authorization

7.1 Getting a token

```
POST /api/v1/auth/token
Content-Type: application/json

{
"email": "admin@api.test",
"password": "Admin#12345"
}
```

Successful response (`200`):

```
{
"accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
"tokenType": "Bearer",
"expiresIn": 3600
}
```

Use it on every protected request:

```
GET /api/v1/auth/me
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

`GET /api/v1/auth/me` returns the current database state, not just the token:

```
{
"userId": "api-admin-user",
"email": "admin@api.test",
"displayName": "API Admin",
"roles": ["Admin", "Customer"],
"shopIds": []
}
```

7.2 Claims

The JWT contains:

Claim	Example	Notes
`sub`	`api-admin-user`	`ApplicationUser.Id` (nvarchar(450)).
`email`	`admin@api.test`	Also the `NameClaimType`.
`name`	`API Admin`	Display name.
`role`	`Admin`, `Customer`	One claim per role. `RoleClaimType` is `role`.
`shop_id`	`9001`	One claim per `ShopUsers` membership. Present for vendors, absent for admins without shop membership.

Claim	Example	Notes
`jti`, `iss`, `aud`, `exp`	—	Standard JWT claims.

`MapInboundClaims = false` keeps these names exactly as issued; the handler does not rewrite them to the legacy WS-Fed URIs.

7.3 Roles versus vendor membership

The application has two security roles: `Admin` and `Customer`. A vendor is **not** a role. Vendor access comes from `ShopUsers` membership, which is projected into the `shop_id` claims. This avoids inventing a third role that the MVC application does not know about.

Policy	Requirement	Used by
`ApiAuthenticated`	Any authenticated user	Reserved
`ApiAdmin`	`Admin` role	`GET /api/v1/admin/dashboard`
`ApiVendor`	`Admin` role **or** at least one `shop_id` claim	`GET /api/v1/shops/{id}/dashboard`
`ApiCustomer`	`Admin` or `Customer` role	Reserved for Phase 3

The shop dashboard controller additionally checks that the requested `shopId` is one of the caller's `shop_id` claims, unless the caller is an `Admin`. A vendor token for shop 9001 therefore gets `403` on shop 9002.

7.4 Lockout and account enumeration

The API mirrors `AccountService`:

- An unknown email still runs a dummy PBKDF2 verification, so it takes roughly the

same time as a wrong password.

- A wrong password increments `AccessFailedCount`.
- At 5 failures the account is locked for 15 minutes and the counter is reset to 0.
- Inactive and locked accounts return the same `401` body as a wrong password:

`"Invalid email or password."` This prevents account enumeration.

- A successful login resets the counter, clears the lockout and updates

`LastLoginUtc`.

The API never returns the password hash or the raw SQL user row.

7.5 Token expiry and revocation

Access tokens live for 60 minutes by default. There is no refresh token in Phase 1; the client signs in again. If an account is deactivated, existing tokens still pass signature validation until they expire, but `GET /api/v1/auth/me` checks the database and returns `401` immediately. Phase 2 can add a token version/`securityStamp` check or short-lived tokens with refresh.

---

8. Endpoint reference

All successful responses are JSON with camelCase property names. Null properties are omitted. Errors are RFC 7807 `application/problem+json` (see section 9).

8.1 Products — search

```
GET /api/v1/products?query=shirt&categoryId=7&shopId=3&minPrice=10&maxPrice=80&inStockOnly=true
&sortBy=price_asc&page=1&pageSize=20
```

Parameter	Type	Default	Rules
`query`	string	null	Matched against name, brand and description. Trimmed; blank becomes null.
`categoryId`	int	null	Exact category.
`shopId`	int	null	Exact approved shop.
`minPrice`	decimal	null	Must be ≥ 0.
`maxPrice`	decimal	null	Must be ≥ 0 and ≥ `minPrice`.
`inStockOnly`	bool	`false`	Only variants with stock > 0.
`sortBy`	string	`relevance`	`relevance`, `price_asc`, `price_desc`, `rating`, `newest`. Unknown values return 400.
`page`	int	`1`	Values below 1 become 1.
`pageSize`	int	`20`	Clamped to 1–100.

Only published, non-deleted products from approved shops are returned. The response is a paged envelope:

```
{
"items": [
{
"productId": 42,
"name": "Cotton Shirt",
"slug": "cotton-shirt",
"brand": "Example",
"isFeatured": true,
"categoryId": 7,
"shopId": 3,
"shopName": "Beirut Threads",
"shopSlug": "beirut-threads",
"minPrice": 24.99,
"totalStock": 18,
"avgRating": 4.6,
"reviewCount": 12
}
],
"page": 1,
"pageSize": 20,
"totalCount": 1,
"totalPages": 1
}
```

`minPrice` is the lowest active variant price. `totalStock` is the sum of active variant stock. `avgRating` and `reviewCount` come from approved reviews.

8.2 Products — detail

```
GET /api/v1/products/42
```

```
{
"productId": 42,
"name": "Cotton Shirt",
"slug": "cotton-shirt",
"brand": "Example",
"shortDescription": "A soft everyday shirt.",
"description": "Long description ...",
"categoryId": 7,
"shopId": 3,
"shopName": "Beirut Threads",
"shopSlug": "beirut-threads",
"reviewCount": 12,
"avgRating": 4.6,
```

```
"variants": [
{
"productVariantId": 101,
"sku": "SHIRT-M",
"name": "Medium",
"price": 24.99,
"compareAtPrice": 29.99,
"stockQuantity": 10,
"lowStockThreshold": 2
}
],
"images": [
{
"productImageId": 501,
"url": "/uploads/products/cotton-shirt.jpg",
"altText": "Cotton shirt",
"displayOrder": 0,
"isPrimary": true
}
]
}
```

The procedure returns four result sets: product, variants, images, rating. The repository reads them in that order. Inactive variants and images are excluded.

8.3 Categories

```
GET /api/v1/categories
GET /api/v1/categories?includeInactive=true
```

Returns an array ordered by `DisplayOrder`, then name:

```
[
{
"id": 7,
"shopId": null,
"isGlobal": true,
"name": "Fashion",
"slug": "fashion",
"description": "Clothing and accessories",
"parentId": null,
"displayOrder": 1,
"isActive": true,
"commissionRate": null
}
]
```

`includeInactive=true` is honoured **only** for an authenticated `Admin`. Anonymous callers always receive active categories, even if they pass the flag.

8.4 Shops

```
GET /api/v1/shops
GET /api/v1/shops/3
```

Only approved shops are public. The list returns the list shape; the detail endpoint adds contact, shipping and lifecycle fields. Nulls are omitted.

```
[
{
"id": 3,
"name": "Beirut Threads",
"slug": "beirut-threads",
"description": "Lebanese-made clothing.",
"tagline": "Wear local.",
```

```
"status": 1,
"city": "Beirut",
"country": "Lebanon",
"logoUrl": "/uploads/shops/beirut-threads/logo.png",
"bannerUrl": "/uploads/shops/beirut-threads/banner.jpg",
"isFeatured": true,
"themeKey": "postal-classic",
"currency": "USD",
"commissionRate": 0.10,
"createdUtc": "2026-01-15T10:00:00Z",
"approvedUtc": "2026-01-16T09:30:00Z"
}
]
```

Shop detail additionally returns `contactName`, `phone`, `email`, `website`, `addressLine1`, `postalCode`, `pickupLocation`, `flatShippingRate`, `freeShippingThreshold`, `suspendedUtc` and `rejectedUtc` when present.

A pending, rejected, suspended or closed shop returns `404` from the public endpoint. This is intentional: the public catalogue does not reveal non-public shop names.

8.5 Platform dashboard (admin)

```
GET /api/v1/admin/dashboard?days=30
Authorization: Bearer <admin token>
```

`days` defaults to 30; values below 1 become 30 and values above 3650 become 3650.

```
{
"days": 30,
"gmv": 125430.50,
"ordersPlaced": 842,
"activeShops": 37,
"shopsAwaitingApproval": 4,
"listingsAwaitingReview": 11,
"reviewsAwaitingModeration": 6
}
```

Field	Meaning
`gmv`	Gross merchandise value of paid-or-later orders in the window.
`ordersPlaced`	Orders placed in the window.
`activeShops`	Approved, non-closed shops.
`shopsAwaitingApproval`	Shops with `Status = PendingApproval`.
`listingsAwaitingReview`	Published products with `ModerationStatus = Pending`.
`reviewsAwaitingModeration`	Reviews with `Status = Pending`.

8.6 Shop dashboard (admin or shop member)

```
GET /api/v1/shops/3/dashboard?days=30
Authorization: Bearer <vendor or admin token>
```

```
{
"days": 30,
"figures": {
"ordersPlaced": 42,
"ordersPaidOrLater": 36,
"ordersCancelled": 2,
"revenue": 8420.75,
"lowStockVariants": 3,
"listingsAwaitingReview": 1
},
"topProducts": [
```

```
{ "productName": "Cotton Shirt", "unitsSold": 120, "sales": 2998.80 }
]
}
```

`revenue` counts paid-or-later orders. `lowStockVariants` counts active variants at or below their `LowStockThreshold`. `topProducts` is ordered by sales descending and limited by the procedure.

---

9. Errors

Every error is `application/problem+json`. The body includes `traceId`; quote it when reporting an issue. The API never returns HTML error pages.

Example `404`:

```
{
"title": "Not found",
"status": 404,
"detail": "Product not found or not published.",
"instance": "/api/v1/products/999999",
"traceId": "0HN7..."
}
```

Example `400` from automatic model validation:

```
{
"title": "Validation failed",
"status": 400,
"detail": "One or more fields are invalid.",
"instance": "/api/v1/auth/token",
"traceId": "0HN7...",
"errors": {
"Email": ["The Email field is not a valid e-mail address."]
}
}
```

Status	When
`200`	Success.
`400`	Invalid query/body, unknown `sortBy`, negative price, `minPrice > maxPrice`.
`401`	Missing/expired/invalid token; wrong email or password; inactive or locked account.
`403`	Authenticated but not an admin, or a vendor requesting another shop's data.
`404`	Product/shop not found or not public.
`409`	Conflict (reserved for Phase 2 writes: duplicate slug/SKU, insufficient stock, invalid state transition).
`429`	Rate limit exceeded.
`500`	Unexpected error. The detail is generic; the real exception is in the API log with the same `traceId`.
`503`	SQL unreachable, login failure, deadlock, or `Api:Enabled = false`.
`504`	SQL command timeout.

The `ApiExceptionFilter` logs unexpected exceptions once at `Error` level. Known business errors are not logged as errors.

---

10. Security

10.1 Transport

- `UseHttpsRedirection()` is enabled.
- Terminate TLS at IIS or a reverse proxy. Add HSTS at the proxy if the API is

internet-facing; the API does not add it itself, so local HTTP development is not forced to HTTPS.

- If the API sits behind a proxy, configure forwarded headers with **known** proxies

so rate limiting sees the real client IP. Do not trust arbitrary `X-Forwarded-For` values.

10.2 Headers

Every response includes:

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer
Permissions-Policy: camera=(), microphone=(), geolocation=()
```

10.3 CORS

`Api:Cors:AllowedOrigins` is empty by default, which means no cross-origin browser access. Add explicit origins for a separate front end:

```
{
"Api": {
"Cors": {
"AllowedOrigins": [ "https://shop.example.com", "https://admin.example.com" ]
}
}
}
```

The policy allows any header and method for those origins. It does not use credentials, so cookies are not part of the API design.

10.4 Rate limiting

One fixed window per token subject (`sub`), or per remote IP for anonymous callers. Default: 100 requests per 60 seconds, no queue. A rejected request returns `429` problem+json. Size the limit for the real traffic and make sure the proxy/forwarded headers are configured, otherwise all anonymous callers may share one partition.

10.5 SQL safety

- Every call is a parameterised stored-procedure call. No string concatenation of

user input.

- The search procedure uses static SQL and `CASE`-based ordering; it does not build

dynamic SQL.

- `SqlErrorTranslator` maps known SQL errors to safe messages. Unexpected SQL errors

become a generic `500` or `503`; the raw message stays in the server log.

- The API login should have only `CONNECT` and `EXECUTE` (see 5.5).

10.6 Secrets

- `appsettings.Local.json` is optional and loaded explicitly. Keep it out of source

control.

- The connection string and signing key are never logged.
- The API does not print tokens or passwords. The smoke script also avoids printing

them.

- The Development signing key is public and is rejected in Production.

10.7 Swagger

Swagger is registered only in Development. Do not enable it in Production. If you need OpenAPI in Production for a gateway, generate the JSON at build time instead of exposing the UI.

---

11. Testing

11.1 Unit and contract tests

```
dotnet test Tests\Kwikby_MP.Api.Tests\Kwikby_MP.Api.Tests.csproj -p:RemoveTestHostAppHost=true
```

45 tests cover:

- `ProductQuery` normalisation, paging clamps, sort whitelist and price validation.
- `SqlErrorTranslator` mappings for all documented custom and built-in error numbers.
- `JwtTokenService` claims (`sub`, `email`, `role`, `shop_id`), issuer/audience and

expiry.

- `ApiOptions` production key rules and default fills.
- SQL contract tests: every expected `usp_*` exists and its parameter names match the

repository calls. These are skipped unless `API_TEST_SQL` is set.

Run the SQL contract tests against the scratch database:

```
$env:API_TEST_SQL = 'Data Source=.;Initial Catalog=MVStores_ApiTest;Integrated
Security=True;TrustServerCertificate=True'
dotnet test Tests\Kwikby_MP.Api.Tests\Kwikby_MP.Api.Tests.csproj -p:RemoveTestHostAppHost=true
```

11.2 Real-process smoke test

`docs/api-smoke.ps1` starts the API (with `-StartApi`), waits for `/health/live`, and checks:

- health live/ready
- product search, product detail, missing product, problem+json content type
- categories (active visible, inactive hidden, anonymous `includeInactive` ignored)
- shops (approved visible, pending hidden, detail 404 for pending)
- admin and vendor sign-in, `/auth/me`, roles and `shopIds`
- `401` for missing token and wrong password; `400` for invalid login body
- platform dashboard values and shop dashboard values
- `403` for a vendor reading another shop and for a vendor reading the platform

dashboard

- `401` for an anonymous platform dashboard

Run it against the scratch database:

```
$env:ConnectionStrings__MultiVendorsStoresConn = `
'Data Source=.;Initial Catalog=MVStores_ApiTest;Integrated
Security=True;TrustServerCertificate=True'
./docs/api-smoke.ps1 -StartApi
```

Or against an already-running API:

```
./docs/api-smoke.ps1
```

Options: `-BaseUrl`, `-AdminEmail`, `-AdminPassword`, `-VendorEmail`, `-VendorPassword`, `-ConnectionString`. It never prints tokens or passwords.

> Note: the in-process `WebApplicationFactory` tests were removed. The only cached > `Microsoft.AspNetCore.Mvc.Testing` version is 8.0.0, whose TestHost response writer > does not implement

`PipeWriter.UnflushedBytes`, which .NET 9's `System.Text.Json` > requires. The real-process smoke script covers the same HTTP pipeline without that > dependency.

11.3 CI

`.github/workflows/ci.yml` restores and builds `Kwikby_MP.sln`, then runs:

- the existing MVC test project (`Tests/WebDS_Test.Tests.csproj`)
- the API test project (`Tests/Kwikby_MP.Api.Tests/Kwikby_MP.Api.Tests.csproj`)

The SQL contract tests run in skip mode on the hosted runner because there is no SQL Server there. Run them locally against `MVStores_ApiTest`, or add a Windows job with SQL Server Express and set `API_TEST_SQL`. The smoke script is not run in CI; it needs a seeded database and a live process.

---

12. Deployment

12.1 Publish

```
dotnet publish Kwikby_MP.Api\Kwikby_MP.Api.csproj
-c Release
-o .\publish\api `
/p:UseAppHost=false
```

12.2 IIS

1. Create an application pool with **.NET CLR version = No Managed Code**.
2. Create a site/application pointing at `publish\api`.
3. Grant the app-pool identity read/execute on the folder.
4. Set environment variables on the app pool or in `web.config`:

`ASPNETCORE_ENVIRONMENT=Production`, `ConnectionStrings__MultiVendorsStoresConn`, `Api__Jwt__SigningKey`.

1. Bind HTTPS. The API redirects HTTP to HTTPS.
2. Point the load balancer's health probe at `/health/ready` (and optionally `/health/live` for liveness).

12.3 Production checklist

- [ ] `05_Deploy.ps1` ran successfully against the target database.
- [ ] SQL contract tests passed against `MVStores_ApiTest`.
- [ ] A database backup exists before the procedure deployment.
- [ ] The API uses a dedicated least-privilege SQL login.
- [ ] `Api:Jwt:SigningKey` is at least 32 random characters and stored in a secret manager, not in source control.
- [ ] `ASPNETCORE_ENVIRONMENT=Production`, so Swagger is off and the dev JWT key is rejected.
- [ ] HTTPS is enforced and the certificate is valid.
- [ ] `Api:Cors:AllowedOrigins` contains only real front-end origins.
- [ ] Forwarded headers are configured if a proxy is in front of the API.
- [ ] Rate limits are sized for expected traffic.
- [ ] Logs are shipped and searched; no secrets are logged.
- [ ] Health probes are wired to the load balancer.
- [ ] A rollback plan exists: keep the previous API build and a copy of the previous

procedure definitions.

12.4 Rollback

The procedure deployment is additive/idempotent; it does not drop tables. To roll back, redeploy the previous procedure definitions and the previous API build. Keep the SQL scripts under version control and tag them with the release.

---

13. Troubleshooting

Symptom	Likely cause	Fix
`503` and `/health/ready` Unhealthy	Connection string empty/wrong, SQL not running, firewall, bad login.	Check the connection string and the health check description.
`401` on `POST /auth/token`	Wrong password, account inactive/locked, or test user not seeded.	Run `06_SeedApiTest.ps1`, or check `IsActive`, `LockoutEndUtc`, `AccessFailedCount`.
`403` on a shop dashboard	Token's `shop_id` does not match the requested shop.	Sign in again after changing `ShopUsers`; claims are issued at login time.
`404` on a product/shop	Not published/approved, or wrong ID.	Public endpoints hide non-public rows. Check `Products.IsPublished`, `Shops.Status`.
`429`	Rate limit exceeded.	Wait for the window, or raise `Api:RateLimit:PermitLimit`.
`500` with `db.error`	Unexpected SQL error.	Read the API log for the same `traceId`; the client body is intentionally generic.
`504`	SQL command timeout.	Raise `Api:CommandTimeoutSeconds` or optimise the procedure/index.
`Invalid object name 'usp_...'`	Procedures not deployed to the target database.	Run `05_Deploy.ps1` against that database.
`The SELECT permission was denied`	Missing execute/select permission.	Grant `EXECUTE` on the schema or the specific procedures.
Swagger returns 404	Not Development, or wrong URL.	Set `ASPNETCORE_ENVIRONMENT=Development` and open `/swagger`.
`MSB3021` / file locked during build	The API is still running.	Stop the process, then rebuild.
`dotnet test` testhost apphost error	Local test-host packaging issue.	Add `-p:RemoveTestHostAppHost=true`.
`PipeWriter ... UnflushedBytes`	Old `Mvc.Testing` 8.0 against .NET 9.	Not used; use `docs/api-smoke.ps1` instead.

---

14. Phase status and next steps

Phase	Scope	Status
0	Separate API project, configuration, JWT auth, error handling, health checks, SQL deployment, tests.	**Done**
1	Public product search/detail, categories, shops, sign-in/me, admin and shop dashboards.	**Done**
2	Admin/vendor writes: product and category CRUD, shop approval, moderation, stock adjustment endpoint, order state transitions.	**Done**
3	Commerce read/status: orders, customers, coupons, reviews, returns, payouts, audit. Order placement via hybrid services still pending.	**In progress**
4	Hardening: refresh tokens, token revocation, OpenTelemetry, idempotency keys, CI with a SQL Server job, load tests.	Not started

Phase 2 should keep the same patterns:

- one stored procedure per write, with the custom `52xxx` error numbers
- a repository method with parameters, never string-built SQL
- a controller action with a policy (`ApiAdmin` or `ApiVendor`) and a scope check
- a contract test for the procedure and parameters
- a smoke-test assertion for the new endpoint

Phase 3 should stay in C#: the existing order/payment/ledger services are already unit-tested, and moving them to T-SQL would duplicate business rules.

---

15. Quick reference

```
# 1. Build everything
dotnet build Kwikby_MP.sln

# 2. Create/refresh the scratch database and seed it
./Database/06_SeedApiTest.ps1

# 3. Run the API tests (including SQL contract tests)
$env:API_TEST_SQL = 'Data Source=.;Initial Catalog=MVStores_ApiTest;Integrated
Security=True;TrustServerCertificate=True'
dotnet test Tests\Kwikby_MP.Api.Tests\Kwikby_MP.Api.Tests.csproj -p:RemoveTestHostAppHost=true

# 4. Run the real-process smoke test
./docs/api-smoke.ps1 -StartApi

# 5. Run the API for manual testing
dotnet run --project Kwikby_MP.Api\Kwikby_MP.Api.csproj --urls http://localhost:5269
```

Test accounts created by `06_SeedApiTest.ps1` (scratch database only):

Email	Password	Access
`admin@api.test`	`Admin#12345`	Admin + Customer, platform dashboard, any shop dashboard
`vendor@api.test`	`Vendor#12345`	Customer + shop 9001 membership, shop 9001 dashboard

---

16. Phase 2 and Phase 3 endpoints (delivered)

Phase 2 (catalogue writes) and Phase 3 (commerce) are now implemented on top of the Phase 1 foundation. Access levels: **A** = Admin, **V** = Vendor (own shop), **C** = Customer, **P** = public.

Catalogue (Phase 2)

Method	Path	Access
GET	`/categories/{id}`	P
POST	`/categories`	A
PUT	`/categories/{id}`	A
DELETE	`/categories/{id}`	A
GET	`/products/admin`	V/A
POST	`/products`	V/A
PUT	`/products/{id}`	V/A
DELETE	`/products/{id}`	V/A
PATCH	`/products/{id}/publish`	V/A
PATCH	`/products/{id}/moderation`	A
GET/POST	`/products/{id}/variants`	P / V/A
PUT/DELETE	`/products/variants/{variantId}`	V/A
GET/POST	`/products/{id}/images`	P / V/A
DELETE	`/products/images/{imageId}`	V/A

Shops and inventory (Phase 2)

Method	Path	Access
POST	`/shops`	C
PUT	`/shops/{id}`	V/A
POST	`/shops/{id}/status`	A
GET/POST/DELETE	`/shops/{id}/team[/{userId}]`	V/A
GET	`/inventory`	V/A
POST	`/inventory/{variantId}/adjust`	V/A
GET	`/inventory/{variantId}/movements`	V/A

Commerce (Phase 3)

Method	Path	Access
GET	`/orders`	C/V/A (scoped)
GET	`/orders/{id}`	C/V/A (scoped)
PATCH	`/orders/{id}/status`	V/A
GET	`/customers`, `/customers/{id}`	A
GET/POST/PUT/DELETE	`/coupons[/{id}]`	A
GET	`/reviews`	A
PATCH	`/reviews/{id}/status`	A
DELETE	`/reviews/{id}`	A
GET	`/returns`	C/V/A (scoped)
PATCH	`/returns/{id}/status`	V/A
GET	`/payouts`	V/A
PATCH	`/payouts/{id}/status`	A
GET	`/audit`	A

Order placement, payment capture and the vendor ledger still run through the existing C# services (hybrid mode). The API exposes the read and status-transition surface above; a checkout endpoint that calls

`IOrderService` is the next step in Phase 3.